

WS-VSW

Vulnerability Risk Assessment Workshop

Build the Scanner. Score the Risk. Model the Threats. Brief Leadership.

STUDENT PACKET

Capstone Workshop | Duration: all day | 3 Parallel Tracks

Prerequisites: Python Foundations (functions, file I/O, data structures, loops) & Cybersecurity
Foundations Lessons 1–18

1. Overview

You've just joined the security engineering team at a mid-size organization as a junior security analyst. Your first assignment: the organization has never had a formal, repeatable process for turning raw vulnerability scan data into a prioritized, business-aware risk picture — and leadership wants a briefing by end of day. Today, you'll build the tool that does it.

This workshop mirrors real security-engineering work: internal teams routinely write tooling that ingests scanner output, enriches it with threat intelligence (CVSS, EPSS, exploit availability), and layers on business context to decide what actually gets fixed first — because a scanner's severity rating alone never tells the whole story.

Learning Objectives

Phase	You will practice...	Lesson topic(s) exercised
1	Reading service/version data and matching it against a vulnerability signature feed	Vulnerability management, Cryptography (TLS/cert findings)
2	Combining CVSS, EPSS, and business-context factors into a defensible risk score	Risk management, Asset management, Data protection
3	Reassessing prioritization when new threat intelligence arrives	Vulnerability management, Attack methodologies
4	Walking a system architecture for threats and mapping them to controls	Architecture models, IAM, Governance, Compliance/audits, Third-party risk
5	Translating technical findings into a leadership-ready briefing	Security communication

2. Workshop Structure & Timeline

Phase	Activity	Time
0	Setup & track briefing	30 min
1	Build the mock vulnerability scanner	80 min
2	Build the contextual risk calculator	80 min
2.5	Generate the risk heat map	30 min
3	Threat intel update & re-score (curveball)	45 min
4	Threat modeling workshop	90 min
5	Executive communication deliverable	45 min

3. Your Track

Complete the Northbridge Financial track (required) and one of the two remaining parallel tracks . Every track uses the same scanner and calculator code — only the underlying data differs. Complete the Appendix for your required/chosen tracks only.

Track	Organization	Focus
A	Meridian Health Partners	Healthcare / HIPAA — patient data protection, clinical system uptime
B	Northbridge Financial (REQUIRED)	Financial services / PCI DSS & SOX — cardholder data, core banking integrity
C	Coastal Logistics Co.	Logistics & supply chain — heavier third-party/vendor exposure, lighter regulatory load

4. Environment Setup

Full setup commands (change directory, create a virtual environment, install dependencies) are in **README.md** at the project root — follow QUICKSTART.md if you just want the copy-paste command list without the explanations.

- Your working files are under workshop/common/ (code) and workshop/data/<your_track>/ (data).
- Confirm your virtual environment is active and pytest is available: run `pytest --version` in your terminal.
- Do not modify `test_scanner.py`, `test_risk_calculator.py`, or `test_heatmap.py` — these are your self-check tools, not part of the assignment.
- Do not edit files under `workshop/data/` — that's your track's source data.

Checkpoint habit

Run your test file after every function you finish, not just at the end. A red test right after you write one function is much easier to debug than six functions in and one giant wall of failures.

5. Phase 1 — Build the Mock Vulnerability Scanner

Real scanners (Nessus, OpenVAS, Qualys) work the same basic way at their core: a feed of “signatures” (a service name, a vulnerable version range, and the resulting finding), checked against what's actually running on each host. You're building a simplified version of that matching engine.

Predict-then-check

Predict before you code

In your own words, write down what you expect `parse_version("7.9p1")` to return, and what you expect `parse_version("10.2")` to return. Then implement the function and check yourself. If you were wrong, write one sentence on why.

Your tasks (in `workshop/common/scanner.py`)

- `parse_version(version_str)` — convert a version string like '1.18.0' into a tuple of ints for comparison.
- `version_is_below(installed_version, threshold_version)` — return True if installed is older than threshold.
- `match_signatures(services_by_asset, signatures)` — find every (asset, service, signature) combination where the service is vulnerable.
- `build_finding(asset, service, signature)` — assemble one Nessus-style finding dict.

Acceptance criteria

GIVEN	WHEN	THEN
a service running nginx 1.16.1	the signature feed lists nginx as vulnerable below 1.21.0	<code>match_signatures</code> returns a match for that asset/service/signature
a service running nginx 1.25.0	the signature feed lists nginx as vulnerable below 1.21.0	<code>match_signatures</code> returns NO match for that service
a matched asset, service, and signature	<code>build_finding</code> is called	the returned dict contains every required key, including host, cve, cvss_base_score, and severity
a completed <code>scanner.py</code>	you run <code>pytest test_scanner.py -v</code>	all tests pass before you run the scanner against real track data

Run it

- From `workshop/common/`, run: `pytest test_scanner.py -v`
- Once green, run: `python3 scanner.py --track <your_track_folder_name>`
- Inspect the printed summary and the generated `scan_results.json` in your track's data folder. Sanity-check: does the finding count and severity mix look plausible for the services you saw in `services.json`?

6. Phase 2 — Build the Contextual Risk Calculator

A scanner tells you what's technically wrong. It doesn't tell you what to fix first. That prioritization call is $\text{Risk} = \text{Likelihood} \times \text{Impact}$ — the same shape used in FAIR-style quantitative risk approaches, simplified for this workshop into a 1–5 scale for each side, multiplied into a 1–25 final score.

Risk factor reference

Likelihood factors — how probable is exploitation?

Factor	1	2–3	4–5
CVSS base score (normalized)	0.0–1.9 — minimal technical severity	2.0–5.9 — moderate	6.0–10.0 — high to critical
EPSS (normalized)	<10% predicted exploitation probability	10–49%	50–100%
Exploit availability	1 = no known exploit code	2–3 = PoC exists, not weaponized	4–5 = weaponized / actively used exploit tooling

Impact factors — how much would it hurt?

Factor	1	2–3	4–5
Asset criticality	1 = insignificant to operations	2–3 = supports some workflows	4–5 = mission-critical
Data sensitivity	1 = public data	2–3 = internal / moderately sensitive	4–5 = highly regulated or critical (PHI, cardholder data, etc.)
Business impact	1 = negligible if compromised	2–3 = noticeable disruption or cost	4–5 = catastrophic — safety, legal, or existential

Why you won't also code an “internet exposure” factor

You might expect a sixth factor for network exposure. We deliberately left it out of the weighted formula: it's already visible to you as each asset's `network_zone` (`internet_facing` / `dmz` / `internal` / `isolated`) and is best used qualitatively during Phase 4 threat modeling, not folded into the numeric score. Think about why scoring it twice — once via CVSS's own attack-vector component and again as a

separate factor — would risk double-counting the same signal.

Predict before you code

Given the current weights (CVSS 0.4 / EPSS 0.4 / exploit availability 0.2 for likelihood), which single factor swinging from 1 to 5 will move the likelihood score the most? Write your prediction, then implement `compute_likelihood` and check it with a scratch calculation.

Your tasks (in `workshop/common/risk_calculator.py`)

- `normalize_cvss_to_scale(cvss_base_score)` — map a 0–10 CVSS score to 1–5.
- `normalize_epss_to_scale(epss_score)` — map a 0–1 EPSS score to 1–5.
- `compute_likelihood(finding, epss_score, exploit_availability)` — weighted combination using `LIKELIHOOD_WEIGHTS`.
- `compute_impact(asset)` — weighted combination using `IMPACT_WEIGHTS`.
- `compute_risk_score(likelihood, impact)` — multiply likelihood × impact, rounded to 2 decimals.

Acceptance criteria

GIVEN	WHEN	THEN
a CVSS base score of 9.8	<code>normalize_cvss_to_scale</code> is called	it returns 5
an EPSS score of 0.03 and an EPSS score of 0.9	<code>normalize_epss_to_scale</code> is called on each	the first returns 1 and the second returns 5
an asset with <code>criticality=5</code> , <code>sensitivity=5</code> , <code>business_impact=5</code>	<code>compute_impact</code> is called	it returns exactly 5.0 regardless of the weight split
a completed <code>risk_calculator.py</code>	you run <code>pytest test_risk_calculator.py -v</code>	all tests pass before you run the calculator against real track data

Run it

- `pytest test_risk_calculator.py -v` — get to all green.
- `python3 risk_calculator.py --track <your_track_folder_name>` — inspect `risk_register.json` and the printed ranking.

Weight experiment (reflection)

Try this, then answer in your packet

Open `risk_calculator.py` and change `IMPACT_WEIGHTS` so `business_impact` carries 0.6 instead of 0.3 (rebalance the others so they still sum to 1.0). Re-run the calculator. Which findings moved in the

ranking, and why? Then change the weights back to the originals before continuing — Phase 3 depends on the default weights.

7. Phase 2.5 — Visualize the Risk Register (Heat Map)

A ranked list is precise but it's not what lands in a five-minute leadership conversation. A risk heat map — Likelihood on one axis, Impact on the other — is the standard visual for this, and you'll generate one from your own data.

Your tasks (in `workshop/common/heatmap.py`)

- `bucket_score(score)` — round a fractional 1.0–5.0 score into an integer grid bucket 1–5, clamped to that range.
- `build_grid(register)` — place every finding into a 5x5 grid cell by its likelihood and impact buckets. The plotting code (`plot_heatmap`) is already written for you — once these two functions are correct, the chart just works.

Acceptance criteria

GIVEN	WHEN	THEN
a score of 3.6	<code>bucket_score</code> is called	it returns 4
a score of 5.9	<code>bucket_score</code> is called	it returns 5 (clamped, not 6)
two findings with likelihood 3.0/3.4 and impact 3.0/2.6	<code>build_grid</code> is called	both findings land in <code>grid[2][2]</code> (bucket 3/3)

Run it

- `pytest test_heatmap.py -v` — get to all green.
- `python3 heatmap.py --track <your_track_folder_name>` — this writes `risk_heatmap.png` into your track's data folder.
- Open the PNG. Do the hottest cells match the top of your `risk_register.json` ranking? They should — if they don't, that's a sign one of your Phase 2 functions has a bug worth revisiting.

Use this in Phase 5

Your executive summary in Phase 5 should reference or embed this heat map — it's the visual equivalent of your top-3-risks list.

8. Phase 3 — Threat Intel Curveball

Your instructor will now hand out (or announce the release of) a threat intelligence bulletin. Read it before doing anything else.

Threat Intel Bulletin

Announcement pending

- Re-run your calculator with the update applied: `python3 risk_calculator.py --track <your_track> --apply-intel-update`
- Compare the new `risk_register.json` ranking against your Phase 2 output.

Reflection — answer in your packet

- Which finding moved the most, and by how much?
- If your team had only sorted findings by CVSS base score, would you have caught this before the bulletin? Why or why not?
- What does this tell you about relying on a single point-in-time severity rating for prioritization?

9. Phase 4 — Threat Modeling Workshop

Your risk register isn't the end of the process — it's the input to a proper threat model. Using your track's top-ranked findings from `risk_register.json`, walk your team's system using STRIDE, then map each threat to a control.

STRIDE quick reference

Category	Question to ask	Example
Spoofing	Can someone pretend to be a legitimate user or system?	Weak/reused credentials on an admin portal
Tampering	Can data or code be altered without authorization?	Unsigned firmware or config updates
Repudiation	Can an action be denied because it wasn't logged?	No audit trail on a privileged database change
Information disclosure	Can data be exposed to those who shouldn't see it?	Unencrypted PHI/cardholder data in transit
Denial of service	Can the system be made unavailable?	Unpatched service crash exploit
Elevation of privilege	Can a user gain more access than intended?	Privilege escalation vulnerability in a DB engine

Worksheet — complete for your top 5 findings

For each of your five highest `risk_score` findings, complete one row:

Finding (host / CVE)	STRIDE category	Threat description	Existing or missing control	Framework mapping

Beyond the top 5 — whole-system questions

- Third-party risk: your track includes a vendor-connected asset. What threats does that trust boundary introduce that a purely internal asset wouldn't have? What contractual or technical control would you recommend?
- Asset management: does anything in your asset inventory's notes field suggest the criticality/sensitivity labels might be stale or wrong? What would you do about it before trusting the risk register built on top of it?

- Architecture: which of your assets sit in the internet_facing or dmz network_zone? For each, is that placement actually necessary, or could a network redesign reduce the attack surface?

10. Phase 5 — Executive Communication Deliverable

Leadership does not want your risk_register.json. They want a one-page briefing they can read in three minutes and act on. Translate your top 3 risks into plain, non-technical language, and make a clear ask.

Executive summary template

- **Headline:** one sentence, plain language, no CVE numbers or CVSS jargon in the headline itself.
- **Top 3 risks:** for each, one line on what could happen and to whom/what, in business terms (e.g., “customer payment data could be exposed”, not “CVE-2019-1068 permits RCE”).
- **Recommended action and rough timeline** for each (e.g., “patch within 5 business days”).
- One sentence on what happens if leadership does nothing.

Security communication reminder

Your audience decides budget and priorities but likely doesn't know what EPSS or CVSS mean. If a term needs a definition to land, either cut it or replace it with a plain-language equivalent.

11. Deliverables Checklist

- scanner.py with all four functions implemented, pytest test_scanner.py -v fully green
- risk_calculator.py with all five functions implemented, pytest test_risk_calculator.py -v fully green
- scan_results.json and risk_register.json (baseline) generated for your track
- heatmap.py with both functions implemented, pytest test_heatmap.py -v fully green, and risk_heatmap.png generated for your track
- risk_register.json regenerated with --apply-intel-update, plus your Phase 3 reflection answers
- Completed STRIDE worksheet (top 5 findings) plus whole-system question answers
- One-page executive summary
- Weight-experiment reflection answer (Phase 2)

Appendix A — Track A: Meridian Health Partners

Healthcare organization operating under HIPAA. Note the appointment scheduler: mission-critical to daily operations, but it holds no clinical data — criticality and data sensitivity deliberately diverge here.

Asset ID	Hostname	Zone	Crit.	Sens.	Bus. Impact	Notes
MHP-01	ehr-db-01	internal	5	5	5	Primary electronic health record database.
MHP-02	patient-portal-web	internet_facing	4	5	4	Patient-facing portal; PHI display, no storage.
MHP-03	appt-scheduler-web	internet_facing	5	1	4	Appointment scheduling only -- names/times, no PHI/diagnosis data. Mission-critical to daily operations despite low data sensitivity.
MHP-04	lab-results-svc	internal	3	5	3	Internal lab results relay service.
MHP-05	staff-wiki	isolated	1	1	1	Internal documentation wiki, air-gapped segment.
MHP-06	billing-vendor-gateway	dmz	4	4	4	Gateway to third-party medical billing clearinghouse (vendor: ClearPath Billing Svcs).

Zone key: internet_facing = directly reachable from the public internet | dmz = perimeter segment fronting an internal system or third-party integration | internal = internal network only | isolated = separated/air-gapped segment.

Appendix B — Track B: Northbridge Financial

Financial services firm under PCI DSS and SOX. Pay close attention to the legacy reporting service's notes field before you trust its criticality label at face value.

Asset ID	Hostname	Zone	Crit.	Sens.	Bus. Impact	Notes
NBF-01	core-banking-app	internal	5	5	5	Core transaction processing application.
NBF-02	online-banking-web	internet_facing	5	5	5	Customer-facing online banking portal.
NBF-03	legacy-reporting-svc	internal	2	5	2	Legacy monthly reporting job. Asset inventory lists as low-criticality decommission candidate, but investigation shows it still ingests full cardholder data extracts nightly -- inventory record is stale.
NBF-04	marketing-site-web	internet_facing	1	1	1	Public marketing brochureware site, no customer data.
NBF-05	hr-payroll-db	internal	3	4	3	Internal payroll database.
NBF-06	fraud-detection-vendor-api	dmz	4	4	4	API gateway to third-party fraud-scoring vendor (vendor: SentryScore Analytics).

Zone key: internet_facing = directly reachable from the public internet | dmz = perimeter segment fronting an internal system or third-party integration | internal = internal network only | isolated = separated/air-gapped segment.

Appendix C — Track C: Coastal Logistics Co.

Logistics and supply-chain company with a lighter regulatory footprint but heavier reliance on third-party integrations (customs brokers, EDI gateways).

Asset ID	Hostname	Zone	Crit.	Sens.	Bus. Impact	Notes
CLC-01	fleet-tracking-svc	internal	4	2	4	Real-time vehicle/shipment tracking service.
CLC-02	customer-tracking-portal	internet_facing	3	2	3	Customer shipment-tracking portal.
CLC-03	warehouse-mgmt-api	internet_facing	3	2	3	API used by handheld scanners in regional warehouses.
CLC-04	vendor-edi-gateway	dmz	4	3	4	EDI gateway to third-party customs-clearance broker (vendor: PortLink Customs Services).
CLC-05	driver-hr-db	internal	2	4	2	Driver personnel records including license and background check data.
CLC-06	internal-analytics-dash	isolated	1	1	1	Internal-only ops dashboard, isolated VLAN.

Zone key: internet_facing = directly reachable from the public internet | dmz = perimeter segment fronting an internal system or third-party integration | internal = internal network only | isolated = separated/air-gapped segment.